TD2 suite : Classes et constructeurs

Exercice 1 : Mettre en pratique les notions liées aux constructeurs en C++ :

- **Identifier le rôle d'un constructeur.**
- **Écrire un constructeur par défaut.**
- **Écrire un constructeur avec paramètres.**
- **Créer des objets en utilisant différents constructeurs.**

À retenir : Un constructeur porte le **même nom que la classe** et ne possède **pas de type de retour**, même pas void.

À retenir : Le constructeur par défaut est un constructeur **sans paramètres**. Il permet d'initialiser les attributs d'un objet avec des valeurs par défaut lors de sa création.

À retenir : Le constructeur avec paramètres permet d'initialiser les attributs d'un objet avec les valeurs données lors de sa création**. Il possède un ou plusieurs paramètres**.

Travail à faire :

On considère la classe suivante :

```
class Livre {
private:
   int code;
   string titre;
   int pages;
public:
   // Constructeurs à compléter
};
```

1) Ajoutez un constructeur par défaut.
2) Ajoutez un constructeur avec paramètres.
3) Complétez la classe par :

a) Une fonction membre, "prixCopie" qui calcule et retourne le prix de la copie du livre en fonction du nombre de pages.

Remarques :

- Le prix initial de la copie est calculé comme suit :
  - Prix initial = nombre de pages × 0,5
- Une remise est accordée lorsque le nombre de pages est supérieur ou égal à 100 :
  - 10 % du prix initial, si le nombre de pages est compris entre 100 et 200 ;
  - 25 % du prix initial, si le nombre de pages est supérieur à 200.
- Prix de la copie = prix initial – remise

b) Une fonction "**afficher**" qui affiche toutes les informations d'un livre.

4) Ecrire un programme main qui :

a) Crée un objet L1 de la classe Livre en utilisant le constructeur par défaut.

b) Crée un objet L2 de la classe Livre en utilisant le constructeur avec paramètres.

c) Crée dynamiquement un objet L3 de la classe Livre avec new.

d) Affiche le prix de copie de chaque livre.

e) Affiche les informations de chaque livre.

f) Libère la mémoire occupée par l'objet créé dynamiquement avec delete.

Exercice 2 : Mettre en pratique la relation de composition en C++

- **Utiliser un objet comme attribut d'une classe.**
- **Initialiser l'objet membre dans le constructeur.**
- **Comprendre la relation de composition entre deux classes.**

- On considère le classe Point suivante :

```
class Point
{
private:
   double x;
   double y;
public:
   Point(double x = 0, double y = 0);
   void afficher() const;
   double distance(const Point& P) const;
};
```

On veut définir la classe Segment. ***Un segment est composé de deux points qui représentent ses deux extrémités***. On parle alors de relation de composition entre la classe Segment et la classe Point.

Pour implémenter en C++, la **relation de composition** entre Segment et Point, la classe Segment va contenir deux **objets membres** de type Point qui représentent respectivement le point d'origine et le point d'extrémité d'un segment.

À retenir : Lorsqu'un objet d'une classe est utilisé comme attribut dans une autre classe, on parle de composition.

À retenir : Dans une composition :

- L'objet contenu fait partie intégrante de l'objet qui le contient.
- L'objet contenu ne peut pas exister indépendamment de l'objet qui le contient.
  - L'objet contenu est créé lors de la création de l'objet qui le contient, et il est détruit lorsque ce dernier est détruit.

Travail à faire :

1. Définir une classe **Segment** ayant deux attributs de type **Point** : O et E (origine et extrémité du segment).
2. Ajoutez un constructeur avec paramètres, qui reçoit en arguments les coordonnées des deux extrémités du segment :

Segment (double x, double y, double u, double v);

À retenir : Les objets membres doivent être initialisés dans la liste d'initialisation du constructeur de la classe qui les contient.

3. Ajoutez une fonction membre qui calcule et retourne la longueur du segment (la distance entre l'origine et l'extrémité).
4. Ajoutez une fonction membre qui affiche les deux extrémités du segment.

À retenir : En POO, on cherche **à réutiliser ce qui existe plutôt que de le réécrire.** Dans une composition, la classe **doit déléguer** à son objet membre les traitements **qui relèvent de la responsabilité de cet objet.**

5. Ecrire un programme de test (main) qui :
    - Crée un segment **S** dont l'origine est le point (1,2) et l'extrémité est le point (3,4).
    - Affiche ce segment.
    - Affiche la longueur de ce segment.

À retenir : Les **méthodes métier** permettent de manipuler un objet comme une boîte noire. Elles permettent de réaliser les opérations utiles, sans exposer les données internes de l'objet.

A retenir : un getter, permet d'exposer une partie de la structure interne de l'objet, ce qui peut aller à l'encontre d'une abstraction forte.

A retenir : un setter permet de modifier individuellement un attribut. Cela peut contourner l'idée de manipuler l'objet comme une capsule. Ce qui est contraire à l'encapsulation.

Exercice 3 : Mettre en pratique les notions d'agrégation en C++

- **Utiliser un objet comme référence membre d'une classe.**
- **Comprendre la relation d'agrégation entre deux classes.**
- **Initialiser une référence membre dans le constructeur.**

Travail à faire :

Nous voulons gérer les séminaires dans un congrès. Chaque séminaire est animé par un seul animateur avec lequel on négocie le tarif par heure.

- Un **animateur** est caractérisé par un **matricule**(int), un **nom** (string) et **un tarif horaire** (double).

1. Définir la classe **Animateur** avec ses attributs.
2. Complétez la classe **Animateur** par :
    - Un constructeur avec paramètres,
    - Une fonction membre "**afficher**",
    - Une fonction membre à qui on passe en argument le nombre d'heures animés par l'animateur et qui retourne le coût de cet animateur.

Remarque : Le coût d'un animateur est calculé selon l'expression ci-dessous :

coût de l'animateur = nombre d'heures * tarif horaire

- Un **séminaire** est caractérisé par son **numéro** (int), son **thème** (string) et le **nombre d'heures animées** (double).
- Chaque séminaire est animé par un seul animateur, qui peut exister indépendamment du séminaire. En POO, cette situation est représentée par une relation d'**agrégation** entre les classes **Seminaire** et **Animateur**.
- En C++, cette relation peut être implémentée dans la classe **Seminaire**, à l'aide d'un attribut de type référence vers un objet **Animateur**.

On considère donc la classe **Seminaire** suivante :

```
class Seminaire
{
   int num;
   string  theme ;
   Animateur& A ; // une référence vers l'animateur
```

```
    double nbhA ;
public :
  ….
} ;
```

À retenir : Lorsqu'une classe possède comme attribut une **référence vers un objet d'une autre classe**, cela représente une relation d'**agrégation**. Dans ce cas, l'objet référencé **existe indépendamment** de l'objet qui le référence.

3. Ajoutez à la classe **Seminaire**, un constructeur avec paramètres permettant d'initialiser le numéro du séminaire, son thème, l'animateur qui l'anime ainsi que le nombre d'heures animées.

Seminaire(int , const char*, const Animateur& , double) ;

À retenir : Une référence membre doit obligatoirement être initialisé dans la liste d'initialisation du constructeur.

4. Ajoutez les fonctions membres suivantes :
   - Une fonction qui affiche les informations d'un séminaire.
   - Une fonction qui retourne le coût du séminaire,
     - **Remarque** : le cout du séminaire est égal au montant payé à l'animateur du séminaire.

À retenir : Dans une agrégation, la classe **doit déléguer** à l'objet référencé les traitements **qui relèvent de la responsabilité de cet objet**.

5. Ecrire un programme main qui :
   - Crée un animateur A.
   - Crée un séminaire animé par l'animateur A.
   - Affiche les informations du séminaire.
   - Affiche le coût du séminaire.